A Plan Language for Federated Biological Retrieval

Capability-indexed lowering, verdict-carrying steps,
and the arithmetic of partial identifier translation

Kundai Farai Sachikonye
Experimental Bioinformatics, TUM School of Life Sciences
Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
kundai.sachikonye@bitspark.com

August 18, 2026

Abstract

A biological question that spans more than one public database is today answered by a script: a host-language program that interpolates identifiers into query strings, sends them, parses what returns, and interpolates again. The host language knows nothing about the data and the query language knows nothing about control flow, and every failure mode of federated retrieval lives in the seam between them. We give a language that closes the seam by refusing to be a query language at all.

The design rests on one observation. The databases a biologist must cross — a reaction knowledge base with a SPARQL endpoint, a protein resource exposing both SPARQL and a REST interface, a pathway resource exposing only flat-file REST, an ontology available as a downloadable artefact — do not share a query model, and no amount of engineering will give them one. They do share a *result* model: finite sets of identifiers carrying attributes. A language whose terms denote result sets and whose operators denote instructions on what to do with a result is therefore portable across backends in a way that a language whose terms denote graph patterns can never be. We call the resulting object a *plan language*; queries are its leaves and are never written by the user.

We develop the language in four layers. Section 3 fixes a source model in which a backend is a triple of an endpoint, a capability set, and an extraction function, and proves that lowering a plan step is total exactly on the steps whose required capabilities are contained in the source's declared set (Theorem 3.15). Section 4 gives the intermediate representation and shows that a plan admits a static capability check that decides compilability before any request is issued (Theorem 4.5), which converts a class of runtime failures into compile-time diagnostics. Section 5 replaces the boolean answer of conventional query interfaces with a six-valued verdict and proves that the six are pairwise distinguishable by a plan executor while any two of them are conflated by an interface returning rows alone (Theorem 5.7, Theorem 5.8). The verdict starved — a step that failed because an *earlier* step under-retrieved — is reachable only in the federated setting, and we show that it is the characteristic failure of federation rather than an exotic case (Theorem 5.9). Section 6 treats identifier translation, which is where the real losses occur: cross-namespace maps are partial, non-injective and non-confluent, and we give exact composition laws for retention (Theorem 6.4), a proof that translation loss is not recoverable by reordering (Theorem 6.8) and a proof that two plans differing only in translation route may return different sets without either being wrong (Theorem 6.11).

Three further results concern execution. Theorem 7.3 solves budget allocation across heterogeneous sources by a water-filling argument with a single shadow price, and Theorem 7.5 shows that a budget exceeding the sum of minimum step costs is necessary but not sufficient for a plan to complete. Theorem 8.5 shows that structural interpolation — lowering a bound result set into a target language through the language's own value-binding construct rather than through string concatenation — eliminates by construction a family of semantic defects that textual interpolation admits, and we exhibit a documented instance of that family in which two spellings of the same intent against the same endpoint returned counts differing by

more than two orders of magnitude. Theorem 9.2 characterises when a plan may be safely re-executed.

We specify an executable prototype, report its behaviour on offline fixtures across four source kinds, and state plainly what has not been established: the prototype has not been run against any live public endpoint, no claim is made about answer correctness against biology, and the capability declarations are asserted by the source adapter rather than verified against the service.

Contents

Part I
The Setting

1 Introduction

1.1 The script that everybody writes

Consider a question of the kind that arises daily in metabolic research: which enzymes catalyse reactions consuming any member of a given chemical class, and of those, which occur in pathways that also contain a second class. No single public resource answers it. The reaction membership lives in one database, the enzyme–reaction association in a second, the pathway membership in a third, and the chemical class hierarchy in a fourth.

The universal solution is a script. In outline:

Listing 1: The script everybody writes. Not a strawman.

```
compounds = sparql(ENDPOINT_A, TEMPLATE_A % class_iri)
ids       = [row["c"] for row in compounds]
enzymes   = []
for chunk in batches(ids, 50):
    q = TEMPLATE_B % " ".join("<%s>" % i for i in chunk)
    enzymes += sparql(ENDPOINT_A, q)
kegg_ids  = [xref(e) for e in enzymes]
pathways  = [rest_get(ENDPOINT_C, "/link/pathway/%s" % k) for k in kegg_ids]
```

Every practitioner recognises listing 1, and every practitioner dislikes it. The dislike is usually expressed aesthetically. It is not an aesthetic problem. Listing 1 exhibits, in eight lines, five distinct and independently serious defects.

(D1) **The interpolation is textual.** Line 5 constructs a query by string concatenation. Whether the resulting string means what the author intended depends on the target language's parse of a string the author never sees in full. Section 8 exhibits a documented case in which two spellings of one intent, differing in a single line, returned 2 and 397 against the same service on the same day.

(D2) **Failure is one bit wide.** If `compounds` comes back empty, the script cannot distinguish: the class has no members; the endpoint timed out and returned an empty result rather than an error; the query used a construct the endpoint does not support and the endpoint degraded silently; the class identifier was misspelled. Each demands a different repair, and the script sees one empty list.

(D3) **The failure of step n is attributed to step n.** If `pathways` is empty, the proximate cause is very often that `kegg_ids` was short, because `xref` is a partial function and silently dropped two-fifths of its input. The script reports a pathway problem and the analyst debugs the wrong stage.

(D4) **The budget is unstated and unallocated.** There is a rate limit on `ENDPOINT_C`, a timeout on `ENDPOINT_A`, and no representation of either anywhere in the program. The script discovers them by failing.

(D5) **The control flow is invisible to the data layer.** The loop over `batches` exists because a query language has no way to say "and then". The batching constant 50 is a magic number encoding an unstated belief about request-size limits at a service the script does not name.

These are not five instances of sloppiness. They are five consequences of one structural decision — that control flow lives in a host language and data access lives in a query language, and neither can see the other.

1.2 Why the obvious repair fails

The obvious repair is to make the query language expressive enough to hold the whole computation: one large query with unions, optional patterns, subselects and federated service clauses. This is what the SPARQL 1.1 federation extension was designed for [Prud'hommeaux and Buil-Aranda, 2013, Harris and Seaborne, 2013], and it fails for two reasons that are not implementation defects.

First, it is frequently not available. Of the four sources in the motivating question, only two speak SPARQL at all. A pathway resource exposing a flat-file REST interface [Kanehisa and Goto, 2000, Kanehisa et al., 2023] cannot be reached by a service clause, because there is no SPARQL protocol endpoint to address. The federation extension federates SPARQL endpoints, and the biological data landscape is not made of SPARQL endpoints.

Second, where it is available, it concentrates the defects rather than removing them. A single query containing every stage of the computation is a single query whose failure is still one bit wide (D2), whose intermediate results are still invisible (D3), and which is now large enough that the interpolation defect (D1) becomes very hard to see — as section 8 demonstrates on a query of eight lines.

A third repair, the workflow system [Wolstencroft et al., 2013, Köster and Rahmann, 2012, Berthold et al., 2009], supplies exactly the control flow (D5) that the query language lacks. It leaves (D1)–(D4) untouched, because a workflow node that issues a query issues a string that its author assembled, and the workflow engine does not know what the string means.

1.3 What we build instead

We separate the two things the script conflates. A *plan* is a sequence of *steps*; a step names a source, an abstract pattern to send it, and a binding to receive the result; the plan language supplies the control flow and the source adapters supply the concrete requests. The user writes the plan. Nobody writes a query.

Principle 1.1 (Queries are leaves)**.** No construct of the plan language denotes a graph pattern, a triple, a relational clause, or a URL. Every such object is produced by *lowering*, from an abstract step together with a source declaration, and is not addressable by the plan author.

Theorem 1.1 is what makes the language portable across backends that share no query model. Its cost is stated immediately: a plan cannot express a request that its source adapters cannot generate, and section 12 records what that excludes.

Principle 1.2 (A step returns a verdict, not rows)**.** The value of a step is a pair (verdict, payload). The payload is a result set only when the verdict is `answered`. Every other verdict carries a diagnosis and no result set.

Theorem 1.2 is what makes the language diagnostic. Its content is section 5, and the reason it is not a mere convention is theorem 5.8: an interface that returns rows, and signals failure by returning zero of them, cannot represent the distinction at all, whatever discipline is imposed on its callers.

Principle 1.3 (Refusal is a first-class outcome)**.** A plan whose declared requirements exceed what its sources can supply, or whose budget falls below the minimum the plan needs, terminates without issuing any request and reports what was missing.

A system with no refusals declines nothing, and therefore claims everything it is asked. Theorem 1.3 is the operational form of that observation, and theorem 4.5 is the result that makes it enforceable statically rather than by convention.

1.4 Contributions and non-contributions

We claim: a source model under which heterogeneous backends compose (section 3); a capability calculus deciding compilability before execution (theorem 4.5); a six-valued verdict algebra with a proof that the conventional interface conflates its members (theorem 5.7, theorem 5.8); exact laws for retention under partial identifier translation (theorem 6.4) with a proof that reordering does not recover loss (theorem 6.8) and that two translation routes may disagree without either being wrong (theorem 6.11); a budget allocation with a single shadow price (theorem 7.3) and a proof that sufficient total budget is not sufficient (theorem 7.5); a proof that structural interpolation eliminates a defect family that textual interpolation admits (theorem 8.5); and a characterisation of safe re-execution (theorem 9.2).

We do not claim: that plans written in this language return biologically correct answers; that the capability declarations of section 3 are verified rather than asserted; that the retention laws of section 6 have been measured against real cross-reference tables; or that the prototype of section 10 has been exercised against any live service. Section 12 enumerates these and others, and theorem 3.16 states the single assumption on which the largest part of the development rests.

1.5 Conventions

$\mathcal{U}$ denotes a countable universe of identifiers. A *namespace* is a subset of $\mathcal{U}$; namespaces are pairwise disjoint, which is a modelling decision and not a fact about the world (theorem 3.10). Result sets are finite. We write $f : X \rightharpoonup Y$ for a partial function and $\operatorname{dom} f$ for its domain of definition. All complexity statements count *requests* unless stated otherwise, since request count is what a public service rate-limits and what a plan can control; wall-clock time is not modelled, and section 12 records the consequence.

2 Related work and where this sits

Federated query processing over distributed data has a substantial literature concerned principally with *source selection* and *join ordering*: given a query and a set of endpoints, decide which endpoints can contribute and in what order to contact them [Schwarte et al., 2011, Görlitz and Staab, 2011, Acosta et al., 2011, Saleem et al., 2016, Rakhmawati et al., 2013]. That work assumes a single query language across sources and optimises within it. Our setting is complementary: the sources do not share a language, the decomposition is written by the user rather than inferred, and the object of study is what a step reports when it fails rather than how fast it succeeds. The two concerns are orthogonal, and a planner of the kind those papers describe could in principle choose among the routes that section 6 shows to be inequivalent — but it would need a cost model over partial maps that the literature does not supply.

Mediator and wrapper architectures [Wiederhold, 1992, Garcia-Molina et al., 1997, Haas et al., 1997] established the pattern of a per-source adapter presenting a common interface, and capability-based query planning [Levy et al., 1996, Papakonstantinou et al., 1995, Florescu et al., 1999, Yerneni et al., 1999] formalised sources that can answer only some queries, binding-pattern restrictions being the classical instance. Our capability sets (theorem 3.2) are in that lineage, and theorem 4.5 is a static-check analogue of their planning-time feasibility tests. The difference lies in what happens on failure: those systems treat an uncoverable query as a planning failure, and we treat it as a typed verdict that the plan can catch and route around, which is the difference between a compiler error and an exception.

Scientific workflow systems [Wolstencroft et al., 2013, Berthold et al., 2009, Köster and Rahmann, 2012, Di Tommaso et al., 2017] provide the control flow we require, and are widely used for exactly the motivating question. They are host languages with library calls, so the requests they issue are strings assembled by the workflow author, and defects (D1)–(D4) survive unchanged.

A workflow system sequences steps; it does not know what a step means, and so cannot report why one came back empty.

Identifier translation across biological namespaces has its own literature and its own well-documented pathologies [Chen et al., 2005, van Iersel et al., 2010, Wimalaratne et al., 2018, Juty et al., 2012, Ison et al., 2016]. The standard framing is pairwise coverage: what fraction of namespace A maps into namespace B. Section 6 takes the *composition* of several such maps as the object, and derives what survives a chain — a quantity a plan actually depends on and which is not determined by the pairwise coverages alone (theorem 6.7).

Finally, the observation that an empty result and a failure are different, and that conflating them corrupts downstream reasoning, is old in database theory: it is the null-value problem [Codd, 1979, Imielinski and Lipski, 1984, Libkin, 2016] in a new setting. Section 5 is a six-valued refinement specific to federation, and theorem 5.9 identifies a value with no analogue in the single-database case.

Part II
The Language

3 The source model

3.1 Sources differ in kind, not dialect

The design turns on a fact about the data landscape that is easy to state and consequential to accept.

Observation 3.1 (Heterogeneity of kind). Among the public resources a biological question routinely crosses, one finds at least the following four kinds:

(i) a SPARQL protocol endpoint accepting arbitrary graph patterns;

(ii) a resource exposing both a SPARQL endpoint and an independent REST interface, with different coverage and different identifier conventions on the two;

(iii) a REST interface returning flat delimited text, with a fixed finite set of operations and no query language whatsoever;

(iv) an artefact — an ontology file, a mapping table — distributed for local loading, whose query capability is whatever the loader provides.

Kind (iii) is the one that forecloses the obvious designs. A resource whose entire query surface is a fixed family of request shapes such as `/link/{target}/{source}` and `/get/{id}` does not admit a compilation target for arbitrary patterns. It admits exactly the operations it names. Any language that compiles to "a query" must therefore either exclude such resources or define "query" so broadly that the word does no work. We take the second horn and rename: what a source accepts is a *request*, and what shapes the request is a *capability set*.

Definition 3.2 (Capability vocabulary)**.** Fix a finite vocabulary $\mathcal{F}$ of *capabilities*. A capability names a construct that a request may use. Throughout,

$$\mathcal{F} = \{\mathsf{pattern}, \mathsf{path}, \mathsf{bind}, \mathsf{agg}, \mathsf{filter}, \mathsf{neg}, \mathsf{order}, \mathsf{lookup}, \mathsf{link}, \mathsf{batch}, \mathsf{regex}\},$$

where pattern is a conjunctive pattern over the source's vocabulary, path a regular path expression, bind the inline supply of a value set as input, agg an aggregate, filter a scalar predicate evaluated at the source, neg negation, order a sort, lookup retrieval of a record by primary identifier, link retrieval of identifiers in a second namespace related to a given identifier, batch the acceptance of more than one input identifier per request, and regex a string match.

The vocabulary is not canonical and nothing below depends on its particular members; what the development requires is only that $\mathcal{F}$ be finite and that capabilities occupy syntactically independent positions in each concrete request language (theorem 3.14).

Definition 3.3 (Source)**.** A *source* is a quadruple $\mathsf{S} = (\eta_{\mathsf{S}}, \mathrm{cap}(\mathsf{S}), \mathrm{ext}_{\mathsf{S}}, c_{\mathsf{S}})$ where η_{S} is an opaque endpoint handle, $\mathrm{cap}(\mathsf{S}) \subseteq \mathcal{F}$ is the *declared capability set*, $\mathrm{ext}_{\mathsf{S}}$ is the *extraction function* mapping a raw response to a result set, and $c_{\mathsf{S}} : \mathbb{N} \to \mathbb{R}_{>0}$ gives the cost of a request as a function of its input cardinality.

Example 3.4 (Four sources)*.* A reaction knowledge base with a SPARQL endpoint declares

$$\{\mathsf{pattern}, \mathsf{path}, \mathsf{bind}, \mathsf{agg}, \mathsf{filter}, \mathsf{neg}, \mathsf{order}, \mathsf{regex}, \mathsf{batch}\}.$$

A flat-file pathway REST interface declares $\{\mathsf{lookup}, \mathsf{link}\}$ and nothing else — in particular not filter, so a plan wishing to filter a pathway result must retrieve and filter locally, at a cost the plan can see. A locally loaded ontology declares $\{\mathsf{pattern}, \mathsf{path}, \mathsf{filter}\}$ but not batch, since the loader admits one query at a time. A protein resource exposing both a SPARQL endpoint and a REST interface is modelled as *two* sources with different capability sets, for the reason given in theorem 3.5.

Remark 3.5 (A dual-interface resource is two sources)*.* Modelling a resource with both SPARQL and REST access as a single source whose capability set is the union of the two is unsound. The union asserts that some request may use path and link together, and no request can: the two capabilities belong to different interfaces, with different coverage and often different identifier conventions. Capability sets are per-interface, and a plan crossing the two interfaces of one resource crosses two sources and pays two costs.

3.2 Results, and why the result model is the common one

Definition 3.6 (Result set)**.** A *result set* over namespace n is a finite set

$$\mathcal{R} \subseteq n \times (\mathcal{A} \rightharpoonup \mathcal{V})$$

of pairs (identifier, partial attribute map), in which no identifier occurs twice. We write $\mathrm{id}(\mathcal{R}) \subseteq n$ for the projection onto identifiers, and $|\mathcal{R}| = |\mathrm{id}(\mathcal{R})|$.

Making the identifier primary and the attributes secondary is the whole of the portability argument, so it is worth establishing that the choice is available.

Proposition 3.7 (The result model is common where the query model is not)**.** *Each of the four source kinds of theorem 3.1 admits an extraction function into theorem 3.6 that is defined on every syntactically valid response.*

Proof. For kind (i), and for the SPARQL interface of kind (ii), a response is a solution sequence: a list of maps from projected variables to values. Designate one projected variable as the identifier column and the remainder as attributes; collapse duplicate identifiers by merging their attribute maps. The construction is defined on every well-formed solution sequence with at least one projected variable.

For the REST interfaces of kinds (ii) and (iii), a response is one of two shapes. Under lookup it is a record keyed by the requested identifier, yielding the singleton whose identifier is that key and whose attribute map is the record's fields. Under link it is a delimited list of pairs, yielding one element per line whose identifier is the second column and whose attribute map records the first column as provenance. Both are defined on every syntactically valid response.

For kind (iv), the loader's own result form is that of kind (i), and the construction of the first paragraph applies. In each case the construction consumes the entire response, so no information is discarded silently, though information may be relocated into attributes. □

Remark 3.8 (What theorem 3.7 does not say). It does not say the extraction is lossless in the sense that matters. A solution sequence with three projected variables and no functional dependency on any one of them — a genuine ternary relation — is forced into identifier-plus-attributes by choosing a key, and the choice loses the multiplicity of the other two columns. A plan that needs an n-ary relation not functionally determined by a key cannot express it in this model. Section 12 records this as a real restriction rather than a formality, and theorem 3.9 gives the standard, unsatisfying workaround.

Remark 3.9 (The workaround, and what it costs). The standard response to the restriction of theorem 3.8 is reification: introduce a synthetic identifier for each tuple of the n-ary relation and carry the original columns as attributes of it. The result is a result set in the sense of theorem 3.6, so the model accepts it, and nothing is lost in the sense of information content.

What is lost is everything downstream. A synthetic identifier lies in no namespace of any source, so it is outside the domain of every translation map of theorem 6.1; a reified step therefore cannot be the input of a translation step, and the retention arithmetic of theorem 6.4 has nothing to say about it. Nor can it be joined against a source on identity, since no source will have minted the same synthetic key. In effect reification buys expressiveness at the price of federation: the reified set is a terminal value the plan may emit but may not carry across a boundary. We call the workaround standard because it is what one does, and unsatisfying because a language for federated retrieval that answers "express it as something you cannot federate" has not answered.

Remark 3.10 (Disjoint namespaces are a modelling decision). We assume namespaces are pairwise disjoint. In practice they are not: the same lexical string may be a valid accession in two databases, and one database may mint identifiers in another's namespace as aliases. The assumption is discharged in the prototype by prefixing every identifier with its namespace tag at extraction time, which makes disjointness true by construction at the cost of making equality between an identifier and its own alias fail. That failure is visible as a retention loss in section 6 rather than as a silent join miss, which we regard as the better trade, but it is a trade.

3.3 Lowering, faithfulness, and honesty

Definition 3.11 (Abstract request)**.** An *abstract request* is a triple $\rho = (\phi, \mathrm{req}(\rho), \beta)$ where ϕ is a pattern over a declared vocabulary, $\mathrm{req}(\rho) \subseteq \mathcal{F}$ is the set of capabilities that ϕ uses, and β is a possibly empty tuple of result sets to be supplied as input. We write $[\![\rho]\!]_D$ for the *denotation* of ρ over a dataset D: the result set that ρ specifies, defined independently of any source.

Definition 3.12 (Lowering)**.** A *lowering* for source S is a partial function $\mathrm{low}_{\mathsf{S}} : \rho \rightharpoonup \mathcal{W}_{\mathsf{S}}$ into the source's concrete request language. $\mathrm{low}_{\mathsf{S}}$ is *faithful for* $f \in \mathcal{F}$ if for every abstract request ρ with $\mathrm{req}(\rho) \subseteq \{f\} \cup \mathrm{cap}(\mathsf{S})$ and every dataset D the source holds,

$$\mathrm{ext}_{\mathsf{S}}(\eta_{\mathsf{S}}(\mathrm{low}_{\mathsf{S}}(\rho))) \;=\; [\![\rho]\!]_D.$$

Definition 3.13 (Honest declaration)**.** $\mathrm{cap}(\mathsf{S})$ is *honest* if, for every $f \in \mathcal{F}$,

$$f \in \mathrm{cap}(\mathsf{S}) \iff \mathrm{low}_{\mathsf{S}} \text{ is faithful for } f.$$

Assumption 3.14 (Syntactic independence)**.** For each source S there is an injection from $\mathcal{F}$ into a set of pairwise disjoint syntactic positions of $\mathcal{W}_{\mathsf{S}}$, such that the concrete request realising a set of capabilities is the one filling exactly the corresponding positions, and its denotation is the conjunction of the denotations of the parts.

Theorem 3.14 holds for the concrete languages we target — a pattern, a path inside a pattern, a binding block, an aggregate wrapper, a filter clause and a sort modifier are separate positions in SPARQL, and a REST interface with a fixed operation family satisfies it degenerately because each capability corresponds to a distinct operation. It fails for languages in which two constructs interact semantically, and theorem 3.18 records one such interaction that is not hypothetical.

Theorem 3.15 (Lowering is total and faithful exactly on contained requests)**.** *Let* S *be a source whose declaration is honest and which satisfies theorem 3.14. Then for every abstract request* ρ*,*

$$\mathrm{low}_{\mathsf{S}}(\rho) \text{ is defined and faithful} \iff \mathrm{req}(\rho) \subseteq \mathrm{cap}(\mathsf{S}).$$

Proof. ($\Leftarrow$) Suppose $\mathrm{req}(\rho) \subseteq \mathrm{cap}(\mathsf{S})$. By theorem 3.13, $\mathrm{low}_{\mathsf{S}}$ is faithful for each $f \in \mathrm{req}(\rho)$ individually. By theorem 3.14 the capabilities in $\mathrm{req}(\rho)$ occupy pairwise disjoint syntactic positions, so the concrete request filling all of those positions is well formed and lies in $\mathcal{W}_{\mathsf{S}}$; hence $\mathrm{low}_{\mathsf{S}}(\rho)$ is defined. Again by theorem 3.14 its denotation is the conjunction of the denotations of its parts, and by faithfulness for each f separately each part denotes what ρ's corresponding component specifies. The conjunction therefore equals $[\![\rho]\!]_D$, so $\mathrm{low}_{\mathsf{S}}(\rho)$ is faithful.

($\Rightarrow$) Suppose $f \in \mathrm{req}(\rho) \setminus \mathrm{cap}(\mathsf{S})$ for some f. By honesty, $\mathrm{low}_{\mathsf{S}}$ is not faithful for f: there exist an abstract request ρ' using f and a dataset D on which either $\mathrm{low}_{\mathsf{S}}(\rho')$ is undefined or the extracted response differs from $[\![\rho']\!]_D$. In the first case $\mathrm{low}_{\mathsf{S}}$ is undefined on some request using f, and by theorem 3.14 the position that f occupies is unfillable for S, hence $\mathrm{low}_{\mathsf{S}}(\rho)$ is undefined as well. In the second case the position is fillable but its part denotes something other than what ρ specifies, and since by theorem 3.14 the whole denotation is the conjunction over parts, $\mathrm{low}_{\mathsf{S}}(\rho)$ is defined but unfaithful. Either way the left-hand side fails. □

Remark 3.16 (Honesty is an assumption, and the interesting one)*.* Theorem 3.15 is exactly as strong as theorem 3.13, and honesty is asserted by the adapter author, not verified against the service. There are two ways to violate it and they are not symmetric. *Under*-declaration — a capability the source in fact supports faithfully, omitted from $\mathrm{cap}(\mathsf{S})$ — is safe: it costs expressiveness, causes some plans to be refused that could have been answered, and never produces a wrong answer. *Over*-declaration is unsound and invisible: the adapter claims path, the service accepts path expressions but evaluates them under a different semantics or truncates them, the lowering is defined, a response comes back, and it is not the denotation. This is not hypothetical — section 8 exhibits a case in which a public service accepted a construct and returned a count differing from the specified denotation by a factor of 198.5. We therefore treat the capability set as a claim to be minimised rather than a feature list to be maximised, and record in section 12 that no mechanical check enforces this, and that we know of no way to build one short of differential testing against a reference implementation.

Proposition 3.17 (Under-declaration can be forced by the service, not the language)**.** *There exist a source* S*, a capability* f*, and a concrete language* $\mathcal{W}_{\mathsf{S}}$ *such that* $\mathcal{W}_{\mathsf{S}}$ *supports* f *in full generality,* $\mathrm{low}_{\mathsf{S}}$ *emits well-formed requests using* f*, every such request receives a well-formed response, and honesty nonetheless requires* $f \notin \mathrm{cap}(\mathsf{S})$*.*

Proof. Take $f = \mathsf{agg}$ and let S be a SPARQL endpoint operating under a server-side cap M on the number of solutions it will materialise. Aggregation is expressible in SPARQL in full generality, so $\mathcal{W}_{\mathsf{S}}$ supports f. Let $\mathrm{low}_{\mathsf{S}}$ emit a counting aggregate over a pattern ϕ whose unaggregated solution sequence has cardinality $N > M$ on the dataset D the source holds. The endpoint materialises M solutions, computes the aggregate over those, and returns a well-formed response carrying the number M. But $[\![\rho]\!]_D = N \neq M$. So $\mathrm{low}_{\mathsf{S}}$ is defined on ρ and unfaithful for f, and by theorem 3.13 honesty forces $f \notin \mathrm{cap}(\mathsf{S})$ even though the target language supports f and the request succeeded. □

Theorem 3.17 is the reason theorem 3.13 is phrased in terms of the lowering and the service jointly rather than in terms of the target language's grammar. Faithfulness is not a property of a language; it is a property of a construct, a lowering, and a deployment. A capability table derived from a specification is therefore not a capability declaration, and treating it as one is precisely the over-declaration error of theorem 3.16.

Remark 3.18 (Where theorem 3.14 fails)*.* The assumption fails when two constructs interact. In SPARQL, a path expression inside a pattern and an inline value block are formally independent

positions, but the *expansion* of an abbreviated object list happens during translation to the abstract algebra, before evaluation, and interacts with path expressions in a way that surprises authors — as section 8 documents. Where independence fails, theorem 3.15 does not apply and the lowering must be verified for the interacting pair jointly. The prototype handles this by never emitting the abbreviated form (theorem 8.4), which restores independence by removing one of the interacting constructs from the emitted language altogether.

4 The intermediate representation

4.1 Steps

Definition 4.1 (Step)**.** A *step* is a tuple $\sigma = (x, \mathsf{S}, \rho, \beta, b)$ where x is the variable the step binds, S is a source, ρ is an abstract request, $\beta = (y_1, \ldots, y_k)$ is a tuple of variables whose values are supplied to ρ as input, and $b \in \mathbb{R}_{>0} \cup \{\infty\}$ is the step's budget annotation.

The shape $\sigma = \text{source} \times \text{request} \times \text{binding}$ is the whole of the intermediate representation, and it is the reason a source adapter is small: of the five components, only low_{S} and ext_{S} are source-specific, and both are functions of the abstract request alone.

Definition 4.2 (Plan)**.** A *plan* $\mathcal{P}$ is a finite sequence $\sigma_1, \ldots, \sigma_m$ of steps together with a total budget B, such that the variable bound by σ_i is distinct from that bound by σ_j for $i \neq j$, and every variable in β_i is bound by some σ_j with $j < i$. The *dependency graph* $G(\mathcal{P})$ has the steps as vertices and an edge $\sigma_j \to \sigma_i$ whenever $y \in \beta_i$ is bound by σ_j.

By construction $G(\mathcal{P})$ is a directed acyclic graph and the sequence order is one of its topological orders. Nothing below depends on which one.

Listing 2: The question of section 1 as a plan. Compare listing 1: no query text, no batching constant, explicit budget, explicit failure handling.

Example 4.3 (The motivating question as a plan).

```
plan enzymes_in_shared_pathways {
  budget 400 requests

  let acids = from CHEBI
      ask descendants_of("CHEBI:35238")
      within 20

  let rxns  = from RHEA
      ask reactions_consuming(?c)
      with ?c in acids
      within 120
      else fail unresolved

  let ecs   = from RHEA
      ask enzyme_of(?r)
      with ?r in rxns
      within 60

  let kids  = map ecs via ec_to_kegg
      expect partial 0.6

  let paths = from KEGG
      ask link("pathway", ?k)
      with ?k in kids
      within 200
      when starved emit partial

  emit paths with provenance
}
```

The `expect partial 0.6` annotation on the translation step is not decoration: it is the assertion checked by theorem 6.14, and a plan whose translation retains less than the asserted fraction stops rather than proceeding on a silently decimated input — which is defect (D3) of section 1, caught at the step that causes it rather than at the step that suffers it.

4.2 The static capability check

Definition 4.4 (Well-capability)**.** A step $\sigma = (x, \mathsf{S}, \rho, \beta, b)$ is *well-capability* if $\mathrm{req}(\rho) \subseteq \mathrm{cap}(\mathsf{S})$, and additionally $\mathsf{batch} \in \mathrm{cap}(\mathsf{S})$ whenever $\beta \neq ()$ and the step is annotated for batched execution. A plan is well-capability if every step is.

Theorem 4.5 (Static decidability of compilability)**.** *Let $\mathcal{P}$ be a plan over sources with honest declarations satisfying theorem 3.14. Then:*

(a) whether $\mathcal{P}$ is well-capability is decidable in time $O(m \cdot |\mathcal{F}|)$, where m is the number of steps, without issuing any request;

(b) if $\mathcal{P}$ is well-capability then every $\mathrm{low}_{\mathsf{S}_i}(\rho_i)$ is defined and faithful;

(c) if $\mathcal{P}$ is not well-capability then there is a step σ_i and a capability $f \in \mathrm{req}(\rho_i) \setminus \mathrm{cap}(\mathsf{S}_i)$ such that no execution of $\mathcal{P}$ can produce a faithful result for σ_i, and the pair (σ_i, f) is computable within the same bound.

Proof. (a) $\mathrm{req}(\rho_i)$ is determined syntactically from ρ_i by structural recursion over the abstract request, and $\mathrm{cap}(\mathsf{S}_i)$ is a declared finite set; the containment test is a subset test over a fixed finite vocabulary, costing $O(|\mathcal{F}|)$ per step with bitset representation. There are m steps and no step's test depends on any other step's result, so the total is $O(m|\mathcal{F}|)$. Nothing in the test consults η_{S_i}, so no request is issued.

(b) Immediate from theorem 3.15 applied stepwise, the hypotheses of which hold by assumption.

(c) If $\mathcal{P}$ is not well-capability then some step fails the containment test; take the least such i in the sequence order and any witness $f \in \mathrm{req}(\rho_i) \setminus \mathrm{cap}(\mathsf{S}_i)$, both produced by the same scan. By theorem 3.15, $\mathrm{low}_{\mathsf{S}_i}(\rho_i)$ is either undefined or unfaithful. In the first case no request is ever issued for σ_i and it has no result. In the second, a result is produced but differs from $[\![\rho_i]\!]_D$ on some dataset, so it is not faithful. Since the executor's behaviour at σ_i depends on the source only through $\mathrm{low}_{\mathsf{S}_i}$ and $\mathrm{ext}_{\mathsf{S}_i}$, no scheduling choice, retry, or budget allocation changes this. □

Corollary 4.6 (Refusal before contact)**.** *A plan that is not well-capability can be refused, with the offending (step, capability) pair named, before any network request is issued.*

Proof. Immediate from theorem 4.5(a) and (c): the test is decidable without issuing a request, and on failure it computes the witness. □

Theorem 4.6 is theorem 1.3 made operational, and it is the concrete payoff of separating cap from ext in theorem 3.3. It is worth being precise about what it buys. It does not guarantee that a well-capability plan succeeds — section 5 lists four ways it can fail with every capability present, and theorem 7.5 adds a fifth. It guarantees only that one particular family of failures, the family in which a plan asks a source for something it cannot do, is converted from a runtime symptom into a compile-time diagnosis naming its own cause.

Proposition 4.7 (The check is not conservative in the useless direction)**.** *There is no plan that is refused by theorem 4.4 and that could nonetheless be executed faithfully against sources with honest declarations.*

Proof. Suppose $\mathcal{P}$ is refused. Then some step has $f \in \mathrm{req}(\rho_i) \setminus \mathrm{cap}(\mathsf{S}_i)$, and by theorem 4.5(c) no execution produces a faithful result for that step. So the refusal excludes no faithfully executable plan. □

Theorem 4.7 is the converse guarantee to theorem 4.6 and is what distinguishes a capability check from a conservative approximation: the check refuses exactly the plans that cannot work, given honest declarations. Under *dishonest* declarations it inherits the dishonesty in both directions, which is theorem 3.16 again and is the reason that remark, rather than this proposition, is the honest summary of what has been established.

Part III
Verdicts

5 The verdict algebra

5.1 Three independent predicates

Before naming the verdicts, we isolate what a verdict is about. Fix a step $\sigma = (x, \mathsf{S}, \rho, \beta, b)$ executed against a source holding dataset D, with the values of β already computed. Three predicates apply, and the burden of this subsection is that they are logically independent.

Definition 5.1 (The three predicates)**.**

- $\mathrm{Exp}(\sigma)$ holds iff the abstract request ρ is expressible against S: formally, $\mathrm{req}(\rho) \subseteq \mathrm{cap}(\mathsf{S})$.
- $\mathrm{Der}(\sigma)$ holds iff $[\![\rho]\!]_D \neq \emptyset$ on the dataset the source in fact holds, with the supplied bindings β.
- $\mathrm{Ans}_b(\sigma)$ holds iff the concrete request completes within the step's budget b.

Proposition 5.2 (Independence)**.** *The three predicates of theorem 5.1 are logically independent: all eight truth assignments are realisable by some (step, source, dataset, budget).*

Proof. We exhibit the constructions; each varies one predicate holding the others fixed, and the eight cases follow by composing them.

To vary Exp alone: take a request using `regex` and swap the source between one declaring `regex` and one not. Neither the dataset nor the budget changes, so Der and Ans_b are untouched on the branch where the request is issued at all; on the branch where it is not, Der is still a property of D and ρ, both fixed, and Ans_b is vacuously assigned by convention.

To vary Der alone: fix the source and request and vary the dataset between one containing a witness for ϕ and one not. Capability declarations do not depend on D, so Exp is unchanged; a request that completes on one finite dataset completes on the other for a budget exceeding both costs, so Ans_b can be held.

To vary Ans_b alone: fix source, request and dataset, and vary b across the cost of the request. Neither $\mathrm{cap}(\mathsf{S})$ nor $[\![\rho]\!]_D$ depends on b, so the other two are unchanged.

Since each predicate is separately variable with the other two held fixed, and each is two-valued, all eight assignments are realisable. □

Theorem 5.2 is the reason a single boolean cannot carry the outcome of a step: it must report a point in an eight-element space, and one bit indexes two. The verdict set below is the coarsening of that space which distinguishes the cases calling for different repairs.

Remark 5.3 (Only one predicate carries a clock)*.* Exp is a property of the request and the declaration; Der is a property of the request and the dataset; only Ans_b mentions the budget. A system reporting a single outcome cannot say which of the three it means, and a practitioner who reruns a failing step with more time is implicitly betting that the failure was in the third. The bet is unfounded two-thirds of the time, and listing 1 provides no way to check it.

5.2 Six verdicts

Definition 5.4 (Verdict set)**.** $V = \{\mathsf{answered}, \mathsf{empty}, \mathsf{unsupported}, \mathsf{exhausted}, \mathsf{refused}, \mathsf{starved}\}$, assigned to a step by the following rules, applied in order:

(R1) If some $y \in \beta$ has a verdict other than answered, or has verdict answered with a payload whose retention fell below the step's declared expectation, then starved; the diagnosis names the offending predecessor.

(R2) Else if $\mathrm{req}(\rho) \not\subseteq \mathrm{cap}(\mathsf{S})$ then unsupported; the diagnosis names $\mathrm{req}(\rho) \setminus \mathrm{cap}(\mathsf{S})$.

(R3) Else if the plan's remaining budget is below c_{S} evaluated at the input cardinality, then refused; the diagnosis names the shortfall.

(R4) Else issue the request. If it does not complete within b then exhausted; the diagnosis names b and the elapsed cost.

(R5) Else if the extracted result set is empty then empty.

(R6) Else answered, with the result set as payload.

The ordering of (R1)–(R6) is not arbitrary and is not merely an implementation convenience: it is what makes the verdicts *attributive*. Each rule fires only when every earlier cause has been excluded, so a step reporting empty has established that its inputs were adequate, its capabilities sufficient, its budget available and its request completed — and therefore that the emptiness is a fact about the data rather than about the machinery. No rule below (R6) can say that.

Definition 5.5 (Blocker)**.** The *blocker* of a non-answered verdict is

$$\mathrm{blk}(\mathsf{unsupported}) = \mathsf{blocked\text{-}by\text{-}model}, \quad \mathrm{blk}(\mathsf{exhausted}) = \mathsf{blocked\text{-}by\text{-}engine}, \quad \mathrm{blk}(\mathsf{refused}) = \mathsf{blocked\text{-}by\text{-}budget},$$

and empty has no blocker.

Remark 5.6 (empty has no blocker on purpose)*.* Assigning a blocker to empty would assert that something obstructed the step, and (R6) fires exactly when nothing did. A verdict of empty is a successful measurement of an empty set. Conflating it with an obstruction is defect (D2) of section 1 in its purest form, and the discipline that prevents it is that the blocker map is partial.

5.3 Distinguishability, and the one-bit interface

Theorem 5.7 (Six-way distinguishability)**.** *For each pair $v \neq v'$ in V there exist a plan, two source configurations agreeing on everything the plan can observe except as noted, and a step whose verdict is v in the first and v' in the second, such that the executor of theorem 5.4 reports the difference. Consequently the six verdicts are pairwise distinguishable by a plan executor.*

Proof. It suffices to realise each verdict by a configuration differing from a common baseline in exactly one respect, since then any two verdicts are separated by the pair of configurations realising them.

Take as baseline a step whose predecessors all returned answered at full retention, whose source declares every capability the request uses, whose budget exceeds the request cost, and whose dataset contains a witness. By (R1)–(R6) the baseline verdict is answered.

empty: remove the witness from D, changing nothing else. (R1)–(R4) still pass; (R5) fires.

exhausted: restore the witness and set b below the request cost. (R1)–(R3) pass, (R4) fires.

refused: restore b and set the plan's remaining budget below c_{S}. (R1)–(R2) pass, (R3) fires.

unsupported: restore the budget and remove one capability of $\mathrm{req}(\rho)$ from $\mathrm{cap}(\mathsf{S})$. (R1) passes, (R2) fires.

starved: restore the capability and set a predecessor's verdict to any non-answered value, or its retention below the declared expectation. (R1) fires.

Each configuration differs from the baseline in exactly one respect and yields a distinct verdict, and the executor computes the verdict by the stated rules, so it reports each. The six are therefore pairwise distinguishable. □

Corollary 5.8 (A row-returning interface conflates at least four verdicts)**.** *Let I be an interface whose response to a request is a result set, with failure signalled by returning the empty set. Then I cannot distinguish* empty*,* exhausted*,* unsupported *and* starved *in any of the four configurations constructed in the proof of theorem 5.7. No discipline imposed on callers of I repairs this.*

Proof. In each of the four configurations the value I returns is the empty result set: in the empty configuration because the denotation is empty; in the exhausted configuration because the request did not complete and I has no other channel; in the unsupported configuration because the request was not issued or was issued and degraded; in the starved configuration because the inputs were empty and a request over empty inputs has empty denotation. The four responses are equal as values. A caller's behaviour is a function of the values it receives, so no caller distinguishes them, and no discipline on callers changes the values. Hence the conflation is a property of I, not of its use. □

Theorem 5.8 is the formal content of defect (D2), and it is worth being explicit about its scope. It does not say that existing systems are badly engineered. It says that the interface contract they expose — rows, with emptiness overloaded as failure — has insufficient capacity, and that a system built on it cannot recover the distinction downstream however carefully it is written. The repair is not better error handling; it is a wider return type.

5.4 Starvation is the characteristic federated failure

The verdict starved deserves separate treatment, because it is the one with no single-database analogue, and because it is not a corner case.

Proposition 5.9 (Starvation is unreachable in the closed setting and reachable in the federated one)**.** *Consider the restriction of theorem 5.4 to plans of a single step against a fixed corpus — the closed setting. Then:*

(a) no execution assigns starved*, and consequently the blocker* blocked-by-corpus *is emitted by no rule;*

(b) in the federated setting with $m \geq 2$ steps, starved *is assigned on a set of configurations of positive measure in the following sense: for every step σ_i with $\beta_i \neq ()$ and every predecessor σ_j feeding it, there is a dataset perturbation at S_j alone that causes σ_i to report* starved *while σ_i's own source, capabilities and budget are untouched.*

Proof. (a) In a single-step plan, $\beta = ()$, so the antecedent of (R1) is vacuously false and (R1) never fires. Since (R1) is the unique rule assigning starved, the verdict is unreachable; and since blocked-by-corpus is by theorem 5.5 the image of starved alone, no blocker blocked-by-corpus is emitted. The corpus is fixed and given, so its inadequacy manifests as empty under (R5) — an empty answer, correctly reported as such, with no obstruction named.

(b) Let $\sigma_j \to \sigma_i$ be an edge of $G(\mathcal{P})$ with $y \in \beta_i$ bound by σ_j. Perturb the dataset at S_j by deleting the witnesses for ρ_j. Then σ_j reports empty by (R5); its capability set, budget and lowering are unchanged, so no other verdict applies to it. At σ_i, the antecedent of (R1) now holds, since y has a verdict other than answered; (R1) fires and σ_i reports starved naming σ_j. Nothing at S_i was altered. □

Remark 5.10 (Why this is the characteristic case). In the closed setting the corpus is an input: if it lacks the assertions a question needs, that is a fact about the world one reports and does not diagnose. In federation the corpus of step i is not an input but an *artefact of steps* $1..i-1$, and its inadequacy is therefore a diagnosable event with a locatable cause. Theorem 5.9 says that federation makes a previously unreachable diagnosis reachable, and theorem 5.11 below says it is the modal outcome once translation enters. Read together with theorem 5.8: the failure mode that federation newly creates is exactly one of the four that the conventional interface cannot see.

Proposition 5.11 (Starvation dominates once translation enters)**.** *Let $\mathcal{P}$ contain a translation step whose retention is $r < 1$ (theorem 6.2), feeding a step whose per-input hit probability is p independently across inputs. Then for input cardinality N the probability that the downstream step returns a non-empty payload is $1-(1-p)^{rN}$, whereas absent the translation it is $1-(1-p)^{N}$. For p small and rN small the ratio of failure probabilities approaches $e^{-prN}/e^{-pN} = e^{pN(1-r)}$, so the excess failure attributable to translation grows exponentially in the lost fraction $1-r$ times the input size.*

Proof. The translation delivers rN inputs by theorem 6.2. Under independence the downstream step misses on all of them with probability $(1-p)^{rN}$, giving the stated success probability; the untranslated case is $r = 1$. Taking the ratio of the two failure probabilities and using $(1-p)^{k} = e^{k\ln(1-p)} \approx e^{-pk}$ for small p yields $e^{-prN+pN} = e^{pN(1-r)}$. □

The independence hypothesis in theorem 5.11 is false in practice — identifiers that fail to translate are correlated with identifiers that are poorly annotated downstream, so the true excess is larger than the bound and not smaller. We state the independent case because it is the one we can prove, and record the direction of the bias rather than claiming the bound is tight.

5.5 Verdict propagation

Definition 5.12 (Propagation)**.** A plan's verdict is answered if every emitted step is answered; otherwise it is the verdict of the earliest non-answered step in the sequence order, together with the transitive closure of blame along $G(\mathcal{P})$.

Proposition 5.13 (Blame is well defined and terminates at a non-starved step)**.** *For any plan and execution, following the diagnosis of a starved verdict from step to named predecessor terminates, and terminates at a step whose verdict is not starved.*

Proof. By theorem 4.2, every variable in β_i is bound by some σ_j with $j < i$, so the named predecessor is strictly earlier in the sequence order. The chain of blame is therefore strictly decreasing in a well-founded order on a finite set and must terminate. It terminates at a step which is not starved, because a starved step always names a predecessor by (R1) and so is never terminal; and the first step of the plan has $\beta_1 = ()$, so (R1) cannot fire on it. □

Theorem 5.13 is what makes a starved report actionable rather than circular: the analyst follows the chain and arrives at a step whose failure is attributed to a capability, a budget, a timeout, or a genuinely empty dataset — one of the four things one can actually do something about. This is defect (D3) resolved, and it is resolved by the structure of the verdict algebra rather than by a convention that callers are asked to observe.

Part IV
Translation

6 The arithmetic of partial identifier translation

Every federated plan that crosses a database boundary crosses a namespace boundary, and the crossing is performed by a cross-reference map. These maps are the least examined component of federated retrieval and the one where the losses actually accumulate. This section treats them as first-class objects of the language and derives what a plan can and cannot know about a chain of them.

6.1 Maps are partial, non-injective, and non-confluent

Definition 6.1 (Translation map)**.** A *translation map* from namespace n to namespace n' is a relation $\mu \subseteq n \times n'$. We write $\mu(S) = \{v : u \in S,\ (u, v) \in \mu\}$ for the image of a set, $\operatorname{dom}\mu = \{u : \exists v,\ (u, v) \in \mu\}$, and call μ *total* if $\operatorname{dom}\mu = n$, *functional* if each u has at most one image, and *injective* if each v has at most one preimage.

Real cross-reference maps are none of the three. A chemical entity may have no counterpart in a pathway database (partiality); a single ontology class may correspond to several database compounds differing only in protonation state (non-functionality); and several distinct ontology classes may map to one database compound that does not resolve the distinction (non-injectivity).

Definition 6.2 (Retention)**.** The *retention* of μ on a set S is

$$r_\mu(S) \;=\; \frac{|S \cap \operatorname{dom}\mu|}{|S|} \qquad (S \neq \emptyset),$$

the fraction of S that survives translation, and $r_\mu(\emptyset) = 1$ by convention. The *amplification* is $a_\mu(S) = |\mu(S)|/|S \cap \operatorname{dom}\mu|$ when the denominator is nonzero.

Retention and amplification are independent: a map may retain everything and amplify (each input has several images), retain little and not amplify, or both. Reporting only the output cardinality $|\mu(S)|$ conflates them, which is why theorem 6.2 separates them and why the prototype records both.

Proposition 6.3 (Cardinality alone is uninformative)**.** *For any $N \geq 1$ and any output size $M \geq 1$ there exist maps μ_1, μ_2 and a set S with $|S| = N$ such that $|\mu_1(S)| = |\mu_2(S)| = M$ while $r_{\mu_1}(S) = 1$ and $r_{\mu_2}(S) = 1/N$.*

Proof. Let $S = \{u_1, \ldots, u_N\}$. For μ_1, distribute M images across all N inputs so that every input has at least one when $M \geq N$, and when $M < N$ let the inputs share images so that all N lie in the domain and the image has M elements; then $r_{\mu_1}(S) = 1$. For μ_2, let $\operatorname{dom}\mu_2 \cap S = \{u_1\}$ and let $\mu_2(u_1)$ have M elements; then $|\mu_2(S)| = M$ and $r_{\mu_2}(S) = 1/N$. The output cardinalities agree and the retentions differ by a factor of N. □

Theorem 6.3 is the reason a plan cannot detect a translation failure by looking at how much came out. In listing 1 the translation is line 7 and its output size is the only thing the script observes about it; by the proposition, that observation carries no information about whether the translation lost anything. This is defect (D3), and it is not fixable by checking that the output is non-empty.

6.2 Composition

Theorem 6.4 (Retention of a chain)**.** *Let $\mu_1 : n_0 \to n_1, \ldots, \mu_k : n_{k-1} \to n_k$ and let S_0 be finite and non-empty, with $S_i = \mu_i(S_{i-1})$. Then*

(a) the end-to-end retention factorises multiplicatively along the realised sets,

$$\frac{|S_k|}{|S_0|} \;=\; \prod_{i=1}^{k} r_{\mu_i}(S_{i-1})\, a_{\mu_i}(S_{i-1}),$$

with the convention that the product is 0 if any S_{i-1} is empty;

(b) if each μ_i is injective on the realised sets, the surviving fraction *— the fraction of S_0 having at least one image in S_k — satisfies*

$$\rho_{1..k}(S_0) \;\le\; \min_{1\le i\le k} r_{\mu_i}(S_{i-1}),$$

and the bound is attained when each μ_i is in addition functional. Without injectivity the inequality fails, and $\rho_{1..k}(S_0) \le r_{\mu_1}(S_0)$ is all that survives (theorem 6.5);

(c) under the same injectivity hypothesis, $\rho_{1..k}(S_0) \ge 1 - \sum_{i=1}^{k}\big(1 - r_{\mu_i}(S_{i-1})\big)$, and the bound is attained when the loss sets are pairwise disjoint under the realised correspondence. Injectivity is again load-bearing: without it a single lost element of S_{i-1} can extinguish several survivors of S_0 at once, and the bound fails (theorem 6.5).

Proof. (a) By theorem 6.2, $|S_i| = |\mu_i(S_{i-1})| = a_{\mu_i}(S_{i-1}) \cdot |S_{i-1} \cap \operatorname{dom}\mu_i| = a_{\mu_i}(S_{i-1}) r_{\mu_i}(S_{i-1}) |S_{i-1}|$ whenever S_{i-1} is non-empty. Telescoping over $i = 1..k$ and dividing by $|S_0|$ gives the product. If some S_{i-1} is empty then S_k is empty and both sides are zero.

(b) An element of S_0 survives to S_k only if it lies in $\operatorname{dom}\mu_1$, and its image lies in $\operatorname{dom}\mu_2$, and so on. Let $A \subseteq S_0$ be the surviving set and fix a stage i. Each $u \in A$ has at least one trajectory reaching a surviving element of S_{i-1}, so the map sending u to one such element is a function $A \to S_{i-1} \cap \operatorname{dom}\mu_i$. If every μ_j is injective on the realised sets this function is itself injective, whence $|A| \le r_{\mu_i}(S_{i-1})|S_{i-1}| \le r_{\mu_i}(S_{i-1})|S_0|$ and the minimum bound follows. Adding functionality gives each element exactly one trajectory, and taking all losses to occur at the single stage achieving the minimum attains the bound. The case $i = 1$ needs no hypothesis, since survival requires membership of $\operatorname{dom}\mu_1$ and distinct elements of S_0 are distinct there; this is the residual bound $\rho_{1..k}(S_0) \le r_{\mu_1}(S_0)$.

(c) The set of elements of S_0 that fail to survive is contained in the union over i of the sets $D_i \subseteq S_0$ whose trajectory dies at stage i. A trajectory dies at stage i by reaching an element of S_{i-1} outside $\operatorname{dom}\mu_i$, and there are $(1 - r_{\mu_i}(S_{i-1}))|S_{i-1}|$ such elements. Under injectivity on the realised sets, distinct members of D_i reach distinct elements of S_{i-1} — the same injection constructed in (b) — so $|D_i| \le (1 - r_{\mu_i}(S_{i-1}))|S_{i-1}| \le (1 - r_{\mu_i}(S_{i-1}))|S_0|$, the last step because injectivity also forces $|S_{i-1}| \le |S_0|$. A union bound over the k stages gives the stated lower bound, with equality when the D_i are pairwise disjoint. Without injectivity $|D_i|$ is not controlled by $|S_{i-1}|$ at all, since one dead element of S_{i-1} may be the sole continuation of arbitrarily many members of S_0. □

Remark 6.5 (Injectivity is a hypothesis of (b) and (c), not a nicety)*.* The hypothesis in theorem 6.4(b) and (c) cannot be dropped. Take $S_0 = \{u_1, \ldots, u_9\}$ with μ_1 retaining seven of them, sending u_9 to $\{t_9\}$ and u_{10} to $\{t_9, t_{10}\}$ — non-injective, but with amplification $a_{\mu_1}(S_0) = 1$, so the collision is invisible in the factorisation of (a). Let μ_2 retain three of the seven, among them t_9. Then $r_{\mu_1} = 7/9$, $r_{\mu_2} = 3/7$, and four elements of S_0 survive, giving $\rho = 4/9 > 3/7 = \min_i r_{\mu_i}$. Two survivors in S_0 are riding one survivor in S_1, so the surviving set upstream is larger than the surviving set downstream and the minimum is not an upper bound for it.

The failure is non-injectivity specifically, not amplification: $a_{\mu_1} = 1$ throughout the example. A cross-reference table that maps two source entries onto one target entry — routine wherever one resource merges what another distinguishes — is exactly this configuration, so the hypothesis is a real restriction on which chains the two bounds may be applied to, and a plan reporting coverage should measure ρ rather than bound it.

The lower bound (c) fails for the same reason, by a different route. Its union bound counts elements of S_0 whose trajectory dies at stage i by counting the elements of S_{i-1} that lie outside $\operatorname{dom}\mu_i$; when several survivors of S_0 have been merged onto one element of S_{i-1}, a single such loss extinguishes all of them, and the count understates the damage. A two-stage instance: $|S_0| = 8$ with realised sizes $|S_1| = 6$, $|S_2| = 5$ and stage retentions $3/4$ and $5/6$, so (c) predicts $\rho \geq 1 - 1/4 - 1/6 = 7/12$, while the measured surviving fraction is $1/2$. The one element lost at stage two carried two survivors, costing $2/8$ where the bound had budgeted $1/6$. Note that the naïve repair — requiring $|S_{i-1}| \leq |S_0|$ — does not help, since it already holds here.

Sampling two-stage chains over eight elements with each element retained independently with probability 0.8 and given up to three images, and keeping only the non-injective draws, (b) fails for 33.9% of chains and (c) for 2.0% ($n = 5000$). Under injective draws neither fails, as the theorem now requires.

Remark 6.6 (The two bounds are far apart, and the gap is the point). For $k = 3$ and $r_{\mu_i} = 0.8$ throughout, on an injective chain, (b) gives $\rho \leq 0.8$ and (c) gives $\rho \geq 0.4$. The true value depends on whether the three maps lose *the same* entities or *different* ones, and no amount of pairwise coverage data determines which. A plan that needs to know its own coverage must therefore measure the chain, not compute it from published per-map figures — which is what theorem 6.14 makes it do, and what theorem 6.7 says it cannot avoid.

Proposition 6.7 (Pairwise coverage does not determine chain coverage)**.** *There exist two families (μ_1, μ_2) and (μ'_1, μ'_2) with $r_{\mu_i} = r_{\mu'_i}$ for $i = 1, 2$ on a common S_0, whose end-to-end surviving fractions differ.*

Proof. Let $S_0 = \{a, b, c, d\}$, and let both first maps be the identity on $\{a, b, c\}$ and undefined on d, so $r_{\mu_1} = r_{\mu'_1} = 3/4$. Let μ_2 be the identity on $\{a, b\}$ and undefined on c, and let μ'_2 be the identity on $\{b, c\}$ and undefined on a; both have retention $2/3$ on $S_1 = \{a, b, c\}$. The chain through μ_2 retains $\{a, b\}$, a surviving fraction of $2/4$; the chain through μ'_2 retains $\{b, c\}$, also $2/4$. To separate them, extend μ'_2 to be defined on a as well and undefined on both b and c: then $r_{\mu'_2} = 1/3 \neq 2/3$, so instead take $S_0 = \{a, b, c, d, e, f\}$, let $\mu_1 = \mu'_1$ be the identity on $\{a, b, c, d\}$ (retention $4/6$), let μ_2 be the identity on $\{a, b\}$ and μ'_2 the identity on $\{c, d\}$, each of retention $2/4$. Both chains survive 2 of 6. Now instead let μ'_1 be the identity on $\{c, d, e, f\}$ — still retention $4/6$ — with μ'_2 the identity on $\{c, d\}$ of retention $2/4$; the primed chain still survives $\{c, d\}$. Finally take μ''_2 to be the identity on $\{e, f\}$, retention $2/4$ on $S'_1 = \{c, d, e, f\}$: the chain μ'_1 then μ''_2 survives $\{e, f\}$, whereas the chain μ_1 then μ''_2 has $S_1 = \{a, b, c, d\}$ meeting $\operatorname{dom}\mu''_2 = \{e, f\}$ in the empty set, surviving fraction 0. The two chains have identical pairwise retentions $(4/6, 2/4)$ and end-to-end surviving fractions $2/6$ and 0. □

6.3 Order does not recover loss

A natural hope is that a plan losing too much to translation can be repaired by reordering: translate later, filter earlier, cross the boundary at a different point. The hope is false in a precise sense.

Theorem 6.8 (Reordering does not recover translation loss)**.** *Let $\mathcal{P}$ contain a translation step μ and a source step σ whose denotation is a selection π, and let $\mathcal{P}'$ be the plan obtained by exchanging their order where both orders are well formed. Then*

$$\mu(\pi(S)) \subseteq \mu(S) \cap \pi'(\mu(S)) \quad \textit{and} \quad \pi'(\mu(S)) \subseteq \mu(S),$$

where π' is the image selection. In particular neither order retains an element of S that lies outside $\operatorname{dom}\mu$*, so the two orders have the same upper bound $r_\mu(S)$ on surviving fraction, and reordering cannot increase it.*

Proof. Both containments are immediate from monotonicity of images and selections. For the substantive claim, let $u \in S \setminus \operatorname{dom}\mu$. In the order "select then translate", u either fails π and is dropped, or passes π and then has no image under μ; either way it contributes nothing to the output. In the order "translate then select", u has no image under μ and so contributes nothing before π' is applied. Hence in both orders the output is contained in $\mu(S \cap \operatorname{dom}\mu)$, whose preimage in S has size at most $r_\mu(S)|S|$. Since this bound is independent of the order, no exchange increases it. □

Remark 6.9 (What reordering does change). Theorem 6.8 says reordering cannot recover *coverage*. It says nothing about *cost*, which it changes substantially: translating after a restrictive selection sends fewer identifiers across the boundary and so costs fewer requests, whereas translating first may allow a batched request where the other order does not. The planner is therefore free to reorder for cost, and theorem 7.3 governs that freedom — but a plan that reorders hoping to retain more is mistaken, and the language does not offer the operation as a coverage remedy.

6.4 Routes may disagree without either being wrong

Definition 6.10 (Route)**.** A *route* from namespace n to namespace n' is a path in the directed graph whose vertices are namespaces and whose edges are available translation maps. Two routes are *parallel* if they share their endpoints.

Theorem 6.11 (Route divergence measures resolved extent)**.** *Let $R = \mu_k \circ \cdots \circ \mu_1$ and $R' = \mu'_l \circ \cdots \circ \mu'_1$ be parallel routes from n to n', each composed of maps that are* sound *— every pair they assert is a true correspondence — and let $S \subseteq n$. Then:*

(a) $R(S)$ and $R'(S)$ cannot contradict: every element of each is a true correspondent of some element of S, so neither set contains an element the other must exclude;

(b) the symmetric difference $R(S) \bigtriangleup R'(S)$ consists exactly of correspondences resolved by one route and unresolved by the other, so $|R(S) \bigtriangleup R'(S)|$ is a lower bound on the number of correspondences that at least one route fails to resolve;

(c) if every map on both routes is total on the realised sets then $R(S) = R'(S)$, so divergence implies partiality somewhere;

(d) consequently $R(S) \cup R'(S)$ is sound and has surviving fraction at least $\max(\rho_R(S), \rho_{R'}(S))$.

Proof. (a) By soundness of each constituent map and transitivity of true correspondence, every $v \in R(S)$ stands in true correspondence to some $u \in S$, and likewise for R'. A set of true correspondents cannot contradict another such set: contradiction would require one route to assert a pair the other denies, and neither route denies anything — both are existential.

(b) Let $v \in R(S) \setminus R'(S)$. By (a), v is a true correspondent of some $u \in S$, and R' does not produce it, so R' fails to resolve the correspondence (u, v). Symmetrically for the other difference. Each element of the symmetric difference therefore witnesses at least one unresolved correspondence on one route, and distinct elements witness distinct correspondences, giving the bound.

(c) If every map is total on the realised sets, each route computes the full composite correspondence relation restricted to S, and the composite relation is determined by the underlying true correspondence, which is route-independent. So the two images coincide.

(d) The union of two sound sets is sound, and contains each, so its surviving fraction is at least each of theirs. □

Remark 6.12 (The union is not free). Part (d) suggests always taking the union of parallel routes. The cost is that the union costs the sum of the routes' requests, and by theorem 7.3 that budget comes from somewhere. The language exposes `union` over routes precisely so that the trade is written down rather than assumed, and theorem 6.13 shows the syntax.

Listing 3: Parallel routes made explicit. The plan asserts nothing about which is better; it measures the divergence and reports it.

Example 6.13 (Two routes, written down).

```
let direct   = map compounds via chebi_to_kegg
let indirect = map compounds via chebi_to_inchikey
               then via inchikey_to_kegg

let both     = union direct indirect
assert soundness(direct) and soundness(indirect)
emit divergence(direct, indirect) as resolved_extent
```

By theorem 6.11(b) the emitted `resolved_extent` is a lower bound on the correspondences at least one route misses. It is not an error count and must not be reported as one: both routes may be individually correct and the divergence is then a statement about coverage, not about accuracy.

6.5 The retention check

Definition 6.14 (Retention check)**.** A translation step annotated `expect partial` ϵ computes $r_\mu(S)$ on its actual input and, if $r_\mu(S) < \epsilon$, returns starved to its successors with a diagnosis naming μ, ϵ and the observed $r_\mu(S)$. Otherwise it returns answered with the image and records both $r_\mu(S)$ and $a_\mu(S)$ as provenance attributes.

Proposition 6.15 (The check converts a silent decimation into a located verdict)**.** *Let $\mathcal{P}$ contain a translation step σ_t with retention below its declared ϵ, feeding a step σ_d that would otherwise report* empty*. Under theorem 6.14, σ_d reports* starved *and the blame chain of theorem 5.13 terminates at σ_t.*

Proof. By theorem 6.14, σ_t returns a non-answered verdict. By (R1) of theorem 5.4, σ_d reports starved naming σ_t. By theorem 5.13 the chain terminates at a non-starved step; since σ_t's verdict was assigned by theorem 6.14 rather than by (R1), and σ_t names no predecessor, the chain terminates there. □

This is the resolution of defect (D3) in its commonest concrete form. Without the check, the analyst sees an empty pathway result and investigates the pathway database. With it, the executor reports that the translation two steps earlier retained 0.31 against a declared 0.6, and names the map.

Part V
Execution

7 Budget allocation across heterogeneous sources

7.1 Why allocation is forced

The sources of theorem 3.1 differ in cost by orders of magnitude and in kind of limit. A locally loaded artefact is effectively free and bounded by memory; a SPARQL endpoint is bounded by a per-request timeout and an unadvertised fair-use policy; a flat-file REST interface is bounded by an explicit request rate. A plan that ignores this either underuses the cheap sources or exceeds the expensive ones, and listing 1 does both. Since the total request budget is finite and the sources'

yields are different functions of the effort spent on them, allocation is an optimisation problem and not a scheduling convention.

Definition 7.1 (Yield)**.** The *yield* of step i under effort $e_i \geq 0$ is $\gamma_i(e_i)$, where $\gamma_i : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$ is non-decreasing, continuously differentiable, strictly concave, and satisfies $\gamma_i(0) = 0$. Effort is measured in requests.

Strict concavity is the substantive assumption and it is the empirically plausible one: the first requests to a source retrieve the common cases and later requests retrieve progressively rarer ones. It fails for a step whose result is all-or-nothing — a single lookup that either succeeds or does not — and theorem 7.7 records the consequence.

Definition 7.2 (Allocation problem)**.** Given a plan with m steps, total budget B, and per-step minimum costs $c_i > 0$, the allocation problem is

$$\max_{e \in \mathbb{R}^m_{\geq 0}} \sum_{i=1}^{m} \gamma_i(e_i) \quad \text{subject to} \quad \sum_{i=1}^{m} e_i \leq B.$$

7.2 A single shadow price

Theorem 7.3 (Water-filling allocation)**.** *The allocation problem of theorem 7.2 has a unique optimum e^*, and there is a scalar $p^* \geq 0$ — the shadow price of a request — such that for every i,*

$$e_i^* > 0 \implies \gamma_i'(e_i^*) = p^*, \qquad e_i^* = 0 \implies \gamma_i'(0) \leq p^*,$$

and $\sum_i e_i^* = B$ whenever some $\gamma_i'(0) > 0$. Consequently $e_i^* = (\gamma_i')^{-1}(p^*)$ on the support, and p^* is the unique root of $\sum_i \max\bigl(0, (\gamma_i')^{-1}(p)\bigr) = B$.

Proof. The objective is a sum of strictly concave functions, hence strictly concave; the feasible set is a compact convex simplex-like region. A strictly concave function on a compact convex set attains a unique maximum, giving existence and uniqueness.

Form the Lagrangian $L(e, p, \lambda) = \sum_i \gamma_i(e_i) - p(\sum_i e_i - B) + \sum_i \lambda_i e_i$ with $p \geq 0$ the multiplier of the budget constraint and $\lambda_i \geq 0$ those of the non-negativity constraints. Slater's condition holds — the point $e_i = B/(2m)$ is strictly feasible — so the KKT conditions are necessary and sufficient. Stationarity gives $\gamma_i'(e_i^*) - p^* + \lambda_i^* = 0$ for each i. Complementary slackness gives $\lambda_i^* e_i^* = 0$. If $e_i^* > 0$ then $\lambda_i^* = 0$ and $\gamma_i'(e_i^*) = p^*$. If $e_i^* = 0$ then $\lambda_i^* \geq 0$ and $\gamma_i'(0) = p^* - \lambda_i^* \leq p^*$.

If some $\gamma_i'(0) > 0$ then the budget constraint is active at the optimum: were $\sum_i e_i^* < B$, adding $\delta > 0$ to that e_i would remain feasible and, by continuity of γ_i' and $\gamma_i'(0) > 0$, would strictly increase the objective, contradicting optimality. Hence $\sum_i e_i^* = B$.

Strict concavity makes each γ_i' strictly decreasing and therefore invertible on its range, so $e_i^* = (\gamma_i')^{-1}(p^*)$ where positive, and $\sum_i \max(0, (\gamma_i')^{-1}(p))$ is continuous and strictly decreasing in p on the region where it is positive, hence has a unique root at B. □

Corollary 7.4 (Nothing is dropped by a rule)**.** *A step receives zero effort at the optimum if and only if $\gamma_i'(0) \leq p^*$, that is, if and only if its marginal yield at the first request is below the price. No step is excluded by a selection heuristic, a priority ordering, or a source blacklist.*

Proof. Immediate from the two implications of theorem 7.3, which are exhaustive and mutually exclusive given strict concavity. □

Theorem 7.4 matters because the alternative — a hand-written priority list saying which sources to try first — is what listing 1 does implicitly through its statement order, and it is unfalsifiable: one cannot ask of a priority list whether it is right. One can ask of p^* whether the yields that determined it were estimated correctly, which is a question with an answer.

Proposition 7.5 (Sufficient total budget is not sufficient)**.** *Let c_i be the minimum cost at which step i can return a non-empty payload. Then $B \geq \sum_i c_i$ is necessary for a plan to complete with all steps* answered*, and is not sufficient.*

Proof. Necessity: each step must be issued at cost at least c_i to return a non-empty payload, and the costs are additive over steps and drawn from one budget, so any completing execution spends at least $\sum_i c_i$.

Insufficiency: take $m = 2$ with $c_1 = c_2 = 1$ and $B = 2$. Let step 2 consume the output of step 1, and let c_{S_2} be increasing in input cardinality with $c_{\mathsf{S}_2}(k) = k$. If step 1 at cost 1 returns a payload of cardinality 3, then step 2 requires cost 3, but only 1 remains. The plan halts with step 2 reporting refused by rule (R3) of theorem 5.4, despite $B \geq c_1 + c_2$. The obstruction is that c_2 is a minimum over inputs and the realised input is not the minimising one, and no static quantity bounds the realised input without executing step 1. □

Remark 7.6 (The consequence for planning). Theorem 7.5 shows that a purely static allocation can be infeasible at run time through no error of the allocator. Two responses are available. The first is to re-solve theorem 7.3 after each step with the remaining budget and the realised cardinalities, which is correct and costs one convex solve per step. The second is to allocate statically from upper bounds on cardinality, which is cheap and wastes budget in proportion to how loose the bounds are. The prototype takes the first, and section 12 records that we have not established a regret bound for it against the clairvoyant allocation — the natural comparison, which we do not make.

Remark 7.7 (Where concavity fails). A lookup step against a REST interface has an all-or-nothing yield: one request retrieves the record and further requests retrieve nothing. Its yield is a step function, neither strictly concave nor differentiable, and theorem 7.3 does not apply to it. The prototype handles such steps by removing them from the optimisation, charging their fixed cost to the budget first, and solving theorem 7.2 over the remainder — which is correct because their effort is not a decision variable, and which means the shadow price governs only the steps that have a real allocation choice.

8 Structural interpolation

8.1 An instance of the defect

The following is a documented observation and we restate it in full because the argument of this section depends on the details rather than on the headline.

Observation 8.1 (Two spellings, two answers). On 10 August 2026, two requests were issued to a public SPARQL endpoint of a reaction knowledge base, with a JSON results media type. The two requests were byte-identical except for one line. The first, using an abbreviated object list, returned a count of 2. The second, using two separate triple patterns, returned a count of 397. The ratio is 198.5.

Listing 4: Form A. Returns 2. The abbreviation is on line 6.

```
PREFIX rh:    <http://rdf.rhea-db.org/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX chebi: <http://purl.obolibrary.org/obo/CHEBI_>
SELECT (COUNT(DISTINCT ?r) AS ?n) WHERE {
  ?r rdfs:subClassOf rh:Reaction ; rh:status rh:Approved .
  ?r rh:side/rh:contains/rh:compound/rh:chebi ?a , ?o .
  VALUES ?aa { chebi:35238 chebi:37022 }
  VALUES ?ox { chebi:35179 chebi:36147 chebi:133294 }
  ?a rdfs:subClassOf* ?aa .
  ?o rdfs:subClassOf* ?ox .
}
```

Listing 5: Form B. Returns 397. Differs from listing 4 only in that line 6 is written as two patterns.

```
  ?r rh:side/rh:contains/rh:compound/rh:chebi ?a .
  ?r rh:side/rh:contains/rh:compound/rh:chebi ?o .
```

The two forms are semantically equivalent under the specification. An object list is syntax: the grammar admits a comma-separated object list in a predicate-object list, and the abbreviation is expanded during translation to the abstract query — before evaluation, and without a special case for property paths [Harris and Seaborne, 2013]. A conforming implementation must therefore return the same count for both. The difference is not a specification ambiguity.

Remark 8.2 (The control is clean and the confound is named). Both forms were evaluated locally against two independent engines [RDFLib Team, 2026, Tanon, 2026] over a hand-checkable dataset, and both engines returned the hand-computed answer for both spellings, so the equivalence is not merely our reading of the grammar. Separately, the endpoint's loaded ontology carried 204,585 classes against 237,672 in the corresponding published download — a difference of 33,087, or 13.9% — so any comparison of counts *across* the two artefacts is confounded by snapshot skew. That confound does not touch theorem 8.1, in which both counts come from the same endpoint in the same session.

Remark 8.3 (We do not diagnose the service). The cause lies in the endpoint's translation of the abbreviated form and we do not attempt to determine it further. Establishing it would require probing a third party's public service with requests designed to elicit internal behaviour, which is not ours to do. Nothing in this section should be read as a recommendation to do so, and the design consequence below does not depend on knowing the cause.

8.2 The design consequence

Theorem 8.1 is an instance of a family. In each member, a plan author writes a request that is correct under the target language's specification, a deployment interprets one spelling differently from another, and the difference is invisible because it is a difference between two strings the author regards as the same string. Textual interpolation puts the author in exactly this position, because the string that is sent is assembled at run time and never inspected.

Construction 8.4 (Longhand-only lowering)**.** The lowering $\mathrm{low}_{\mathsf{S}}$ emits, for each abstract request, a canonical concrete form fixed by the adapter, in which (i) every predicate-object list is expanded to separate patterns, (ii) every bound result set enters through the target language's own value-binding construct rather than by concatenation into the pattern text, and (iii) no construct is emitted whose expansion interacts with another construct in the sense of theorem 3.18.

Theorem 8.5 (Structural interpolation eliminates the defect family)**.** *Let $\mathcal{D}$ be the family of defects in which two concrete requests with the same denotation under the target specification are assigned different denotations by a deployment, and whose difference lies in the spelling of a construct that theorem 8.4 canonicalises. Then no execution of a plan under theorem 8.4 exhibits a member of $\mathcal{D}$.*

Proof. Fix a plan and an execution. Every concrete request issued is produced by $\mathrm{low}_{\mathsf{S}}$, since by theorem 1.1 no other source of concrete requests exists in the language. By theorem 8.4, $\mathrm{low}_{\mathsf{S}}$ emits a single canonical spelling for each abstract request: the expanded, longhand form with bindings supplied structurally. Membership in $\mathcal{D}$ requires two requests differing in a canonicalised spelling. But for any two abstract requests with the same denotation, the emitted concrete forms are determined by the canonical procedure and therefore agree wherever the canonicalisation applies; and no non-canonical spelling is ever emitted, so no pair of emitted requests differs in one. Hence no member of $\mathcal{D}$ is exhibited. □

Remark 8.6 (Precisely what is and is not eliminated). Theorem 8.5 is a construction-by-exclusion result and its strength is exactly the coverage of theorem 8.4. It eliminates the family $\mathcal{D}$ —

differences attributable to a spelling the lowering canonicalises. It does not eliminate: deployments that misinterpret the canonical form itself; capability over-declaration (theorem 3.16), which is a different defect with the same symptom; or divergence between a service and a specification in constructs the canonicalisation does not touch. The honest statement is that the language removes the author's ability to make one class of mistake, by removing the author's access to the string, and that this is a smaller claim than correctness.

Proposition 8.7 (The elimination is not available to textual interpolation)**.** *Let I be an interface in which the caller supplies the concrete request as a string. Then for any canonicalisation procedure C, there is a caller of I whose issued request is not in the image of C.*

Proof. The caller supplies an arbitrary string and I transmits it. Since C's image is a proper subset of the concrete language — proper because C canonicalises, so at least two distinct strings map to one, and the non-canonical one is outside the image — a caller supplying that non-canonical string issues a request outside the image. No property of I prevents this, because I's contract is to transmit what it is given. □

Theorem 8.7 is why theorem 1.1 is a principle of the language rather than a recommended practice: a recommendation is exactly the thing theorem 8.7 says is unenforceable.

9 Re-execution

A plan is often run more than once — to refresh a result, to recover from a budget exhaustion, to check a suspicious answer. Whether re-execution is safe is not obvious, because a plan mixes sources with different volatility and a retention check compares against a threshold.

Definition 9.1 (Stability)**.** A source S is *stable over an interval* if the dataset it holds is unchanged throughout. A translation map is stable if its relation is unchanged.

Theorem 9.2 (Characterisation of safe re-execution)**.** *Let $\mathcal{P}$ be well-capability with total budget B, executed twice over an interval throughout which every source and every translation map it uses is stable. Then the two executions assign the same verdict and the same payload to every step if and only if both executions allocate every step an effort at least its realised cost. If some step is allocated less in one execution than its realised cost, the two executions may differ, and the difference is confined to that step and its successors in $G(\mathcal{P})$.*

Proof. ($\Leftarrow$) Suppose both executions allocate every step at least its realised cost. We induct along the sequence order. For σ_1, $\beta_1 = ()$, so (R1) does not fire; $\mathrm{req}(\rho_1) \subseteq \mathrm{cap}(\mathsf{S}_1)$ by well-capability, so (R2) does not fire; the allocation covers the realised cost, so neither (R3) nor (R4) fires; the request is issued, and by stability the dataset is the same in both executions, so by faithfulness (theorem 3.15) the extracted result equals $[\![\rho_1]\!]_D$ in both. Hence the verdict and payload agree. For the inductive step, assume all predecessors of σ_i agree across the two executions. Then the antecedent of (R1) has the same truth value in both, and if it is false the same argument as the base case applies, using stability of any translation map involved and the fact that a retention check compares a fixed $r_\mu(S)$ against a fixed ϵ, so it too agrees.

($\Rightarrow$) Suppose some step is allocated less than its realised cost in some execution. Then in that execution rule (R3) or (R4) fires at that step, yielding refused or exhausted; if the other execution allocates at least the realised cost, that step yields answered or empty there. The verdicts differ, contradicting agreement. Hence agreement forces sufficient allocation everywhere.

Confinement: by theorem 5.4, a step's verdict depends only on its own source, capabilities, budget and the verdicts and payloads of its predecessors. If all predecessors of σ_j agree and σ_j is not the differing step, the same argument as above gives agreement at σ_j. So the difference propagates only along edges of $G(\mathcal{P})$ out of the differing step. □

Corollary 9.3 (Budget exhaustion is recoverable, data change is not)**.** *Re-running a plan that halted with* refused *or* exhausted*, under a larger budget and over a stable interval, reproduces every verdict the first execution assigned before the halt and may extend the plan further. Re-running across a change in any source's dataset carries no such guarantee, and the language records source snapshot identifiers in provenance so that the two cases are distinguishable after the fact.*

Proof. The first claim is theorem 9.2($\Leftarrow$) applied to the prefix of steps allocated sufficiently in both executions, which by hypothesis includes every step completed before the halt. The second follows because stability is a hypothesis of theorem 9.2 and the theorem's conclusion is not asserted without it; the provenance recording is a design decision, not a consequence. □

Part VI
Realisation

10 A prototype

10.1 What the prototype is, and what it is not

The prototype is a compiler and executor for HFQ written in Python. It parses plan text into the intermediate representation of theorem 4.2, performs the static check of theorem 4.5, solves the allocation of theorem 7.3, executes the plan against source adapters, assigns verdicts by theorem 5.4, and writes the result — verdicts, payloads, retention figures, allocation, and provenance — as a JSON document.

It is not a benchmark. Every adapter in the prototype resolves against a local fixture or a local engine. No request leaves the machine. This is a deliberate constraint and not a temporary limitation of the implementation: the properties under test are properties of the compiler and the verdict assignment, and a live service can neither confirm nor refute them while introducing a dependency on a third party's availability, policy, and current snapshot. What a live service would test — whether real cross-reference tables have the retention figures one hopes — is a question this paper does not answer and does not claim to (section 12).

10.2 Pipeline

(1) **Parse.** Plan text to a list of steps $(x_i, \mathsf{S}_i, \rho_i, \beta_i, b_i)$, together with the plan budget. The grammar is small: a plan is a budget declaration followed by `let` bindings and a terminal `emit`.

(2) **Resolve.** Each source name is resolved against a registry of adapters, each of which declares $\mathrm{cap}(\mathsf{S})$ as an explicit set of feature symbols. The declaration is data, written by the adapter author, and is the locus of the honesty assumption (theorem 3.16).

(3) **Check.** Compute $\mathrm{req}(\rho_i)$ for each step by structural recursion over the abstract request, and test $\mathrm{req}(\rho_i) \subseteq \mathrm{cap}(\mathsf{S}_i)$. On failure the executor halts before issuing any request and emits a refusal document naming the missing features and the step — theorem 4.6 made operational.

(4) **Allocate.** Solve theorem 7.2 by bisection on the shadow price p, using $\gamma_i(e) = w_i \log(1+e)$ as the default yield, whose derivative $w_i/(1+e)$ inverts in closed form to $e = w_i/p - 1$. Steps with step-function yields are charged first (theorem 7.7).

(5) **Execute.** In sequence order, apply (R1)–(R6). A step reaching (R4) calls its adapter's lowering, which emits the canonical concrete request of theorem 8.4; the adapter evaluates it against its fixture and returns a result set.

(6) **Emit.** Serialise to JSON.

10.3 Adapters

Four adapter kinds are implemented, chosen to exercise the four source kinds of theorem 3.1 rather than to be complete.

- A *graph-pattern adapter* over a local RDF store [RDFLib Team, 2026, Tanon, 2026], declaring {pattern, path, bind, filter, agg}. Its lowering emits longhand triple patterns and supplies bound sets through the value-binding construct, never by concatenation.
- A *lookup adapter* over a local key–value fixture, declaring {lookup, link} and *not* pattern. It is the prototype's stand-in for a flat-file REST interface, and its restricted declaration is what makes theorem 4.5 fire on plans that ask it for a join.
- An *ontology adapter* over a local class hierarchy, declaring {path, lookup} — transitive closure over subsumption but no filtering and no aggregation.
- A *map adapter* implementing translation steps (theorem 6.1), which is the only adapter that reports retention and amplification.

Remark 10.1 (The declarations are the experiment). Because the static check is decided entirely by the declarations, and the declarations are chosen by us, the prototype cannot demonstrate that the check is *right* about any real service. What it demonstrates is that the check is *decidable and cheap*, that refusal precedes contact, and that the verdict assigned to a rejected plan names the missing feature rather than reporting an empty result. Those are the claims of theorem 4.5 and theorem 4.6, and they are properties of the compiler.

10.4 The output document

The JSON document produced by an execution has one entry per step, and the schema is fixed by the theory rather than by convenience:

Listing 6: Shape of a step entry in the emitted JSON. The `blocker` field is absent exactly when the verdict is empty or answered, per theorem 5.5.

```
{
  "step": "kids",
  "source": "CHEBI->KEGG",
  "verdict": "starved",
  "blocker": "corpus",
  "diagnosis": {
    "named_predecessor": "ecs",
    "reason": "retention 0.31 below declared expectation 0.6"
  },
  "retention": 0.31,
  "amplification": 1.14,
  "budget": {"allocated": 60, "spent": 60, "shadow_price": 0.0142},
  "payload": null,
  "provenance": {"snapshot": "fixture-v3", "lowered_form": "canonical"}
}
```

Three features of listing 6 are consequences of results above rather than design taste. The `blocker` field is partial, because theorem 5.5 is partial and empty has no blocker. The `payload` is `null` for every non-answered verdict, because theorem 1.2 says a payload accompanies only answered — and an implementation that returned a partial payload alongside a failure verdict would reintroduce exactly the ambiguity theorem 5.8 identifies. `retention` and `amplification` are recorded separately, because by theorem 6.3 their product — which is all the output cardinality reveals — determines neither.

11 Validation design

We enumerate the checks the prototype performs, in the interest of making it clear which claims of this paper are exercised and which are not. Each is executed against local fixtures.

(V1) **Static check decides without contact.** For a family of plans, half well-capability and half not, verify that the ill-capability plans halt with a refusal document, that the refusal names the correct missing features, and that the adapter request counter is zero. (theorem 4.5, theorem 4.6.)

(V2) **Check cost is linear.** Measure the check's operation count against $m|\mathcal{F}|$ over plans of growing length. (theorem 4.5(a).)

(V3) **Six verdicts are realised.** Construct the six configurations of the proof of theorem 5.7 — each differing from a common baseline in one respect — and verify that the executor assigns six distinct verdicts. (theorem 5.7.)

(V4) **The one-bit interface conflates four.** Run the same six configurations through a deliberately impoverished adapter that returns only a result set, and verify that four of the six are indistinguishable in its output. (theorem 5.8.)

(V5) **Starvation is unreachable when $m = 1$.** Over all single-step configurations in the fixture family, verify that starved is never assigned and blocked-by-corpus never emitted; then verify that the predecessor perturbation of theorem 5.9(b) produces it in a two-step plan.

(V6) **Blame terminates.** For every starved execution, follow the diagnosis chain and verify it terminates at a non-starved step within m hops. (theorem 5.13.)

(V7) **Retention factorises.** Over random partial maps, verify the multiplicative identity of theorem 6.4(a) exactly, and verify that the measured surviving fraction lies within the bounds of (b) and (c). Record how often each bound is attained and how wide the gap is (theorem 6.6).

(V8) **Cardinality is uninformative.** Realise the two maps of theorem 6.3 and verify equal output cardinality with retentions differing by the predicted factor.

(V9) **Pairwise coverage does not determine chain coverage.** Realise the two families of theorem 6.7 and verify identical pairwise retentions with differing end-to-end surviving fractions.

(V10) **Reordering does not recover coverage.** For plans admitting both orders, verify equal surviving fractions and unequal request counts. (theorem 6.8, theorem 6.9.)

(V11) **Routes diverge without contradiction.** Construct parallel sound routes with partial maps, verify that neither contains an element contradicting the other, that the symmetric difference is non-empty, and that making every map total collapses the divergence to zero. (theorem 6.11.)

(V12) **Allocation satisfies KKT.** Solve theorem 7.2 numerically and verify that the marginal yields of all supported steps agree to tolerance, that unsupported steps satisfy $\gamma_i'(0) \leq p^*$, and that the budget binds. (theorem 7.3.)

(V13) **Sufficient budget is not sufficient.** Realise the two-step counterexample of theorem 7.5 and verify a refused verdict despite $B \geq \sum c_i$.

(V14) **Lowering is canonical.** For every abstract request in the fixture family, verify that the emitted concrete form contains no predicate-object list and no interpolated identifier outside a value-binding construct, and that two abstract requests with the same denotation lower to the same string. (theorem 8.4, theorem 8.5.)

(V15) **Re-execution is stable.** Execute each plan twice over an unchanged fixture with sufficient budget and verify identical verdicts and payloads; then re-execute with a budget below one step's realised cost and verify that the difference is confined to that step and its successors. (theorem 9.2.)

We record what these do *not* establish, since the enumeration invites the opposite reading. (V1)–(V15) test the compiler and the executor against the theory. They do not test the theory against the world. No check above would detect a dishonest capability declaration, an inaccurate cross-reference table, or a deployment that misinterprets the canonical form — and the first and third of those are precisely the failure modes theorem 3.16 and theorem 8.6 identify as beyond the reach of the construction.

12 Limitations

Honesty is assumed, never verified. The whole of theorem 4.5 and the greater part of section 5 rest on the assumption that a source's declared capability set is honest (theorem 3.13). Under-declaration costs expressiveness and is safe; over-declaration is unsound and, worse, invisible — an over-declared source accepts a request it will silently degrade, and the degraded answer arrives as `answered` with a payload. We do not know how to verify a declaration without probing the service, and probing a third party's public service to characterise its internals is not something we undertake. This is the single largest gap in the development and we do not minimise it.

Yields are posited, not estimated. Theorem 7.3 assumes strictly concave differentiable yield functions and the prototype supplies $w_i \log(1 + e)$ by default. We have not estimated a yield function for any real source, and the weights w_i are configuration. An allocation optimal for the wrong yields is not optimal. Furthermore theorem 7.7 excludes all-or-nothing steps from the optimisation entirely, so the shadow price governs a subset of the plan.

No regret bound for replanning. Theorem 7.6 adopts re-solving after each step, and we have not bounded its total yield against the clairvoyant allocation that knew the realised cardinalities in advance. This is the natural theoretical question raised by theorem 7.5 and we do not answer it.

Retention laws are unmeasured. Theorem 6.4 is arithmetic and holds of any partial maps. Whether real cross-reference tables have retentions making theorem 5.11 bite, and whether the losses of successive maps are correlated or independent, is an empirical question we raise and do not settle. We note only that the independence hypothesis is false in the direction that makes the bound conservative.

Namespace disjointness is a modelling decision. Theorem 3.10 records that we take namespaces to be pairwise disjoint. Real identifier spaces overlap and are re-used, and a shared identifier appearing in two namespaces would be conflated by the model. We do not know how badly.

Wall-clock time is not modelled. All costs count requests (section 1, conventions). A plan whose requests are few and slow and one whose requests are many and fast are indistinguishable to the allocator. The verdict `exhausted` mentions a clock but the budget does not, and reconciling the two would require a second resource dimension the model does not carry.

The static check is not a completeness result. Theorem 4.5 says a well-capability plan compiles. It does not say an ill-capability plan is unanswerable: a source might answer a request outside its declared capabilities through a route the adapter does not expose. Refusal is conservative, and conservatism is the safe direction, but it is not optimality.

Structural interpolation is a smaller claim than correctness. Theorem 8.6 states this in place and we repeat it here because the result is the one most liable to be over-read. Theorem 8.5 eliminates a family of defects by removing the author's access to the request string. It does not make the request correct, and a deployment that misreads the canonical form is untouched by it.

Nothing has been run against a live service. By construction. The consequence is that every quantitative figure in the prototype's output is a figure about a fixture, and no claim in this paper is supported by measurement of a public biological database.

13 Conclusion

The script of listing 1 is not bad code. It is the best code available under a division of labour in which control flow lives in a host language and data access lives in a query language, with no channel between them. Its five defects are consequences of that division and not of its author.

We have described a language that moves the division. Control flow and data access live together in a plan; the plan author never writes a request; requests are generated by adapters from abstract steps and source declarations. The consequences follow from that single relocation. Because requests are generated, their spelling is canonical and a family of interpolation defects cannot occur (theorem 8.5). Because sources declare capabilities, compilability is decidable before contact and refusal precedes any request (theorem 4.5). Because a step returns a verdict rather than rows, the four outcomes a row-returning interface conflates become distinguishable (theorem 5.8) — including one, starvation, that has no single-database analogue and becomes reachable only in federation (theorem 5.9). Because translation is a first-class step, its retention is measured rather than assumed, and the chain laws say what a plan can and cannot infer about its own coverage (theorem 6.4, theorem 6.7) — including that reordering will not help (theorem 6.8) and that two sound routes may disagree without either being wrong (theorem 6.11). Because the budget is declared, allocation is an optimisation with a single shadow price rather than a priority list (theorem 7.3), though a sufficient total is still not sufficient (theorem 7.5).

What the development does not have is a way to check the one thing it most depends on. A capability declaration is an assertion by an adapter author, and an over-declaration is unsound and silent. Every static guarantee in this paper is conditional on that assertion. Whether the condition can be discharged without probing the services in question — which is not ours to do — is the question we would most like answered and the one we leave open.

References

Maribel Acosta, Maria-Esther Vidal, Tomas Lampo, Julio Castillo, and Edna Ruckhaus. ANAPSID: An adaptive query processing engine for SPARQL endpoints. In *The Semantic Web – ISWC 2011*, volume 7031 of *Lecture Notes in Computer Science*, pages 18–34. Springer, 2011. doi: 10.1007/978-3-642-25073-6_2.

Michael R. Berthold, Nicolas Cebron, Fabian Dill, Thomas R. Gabriel, Tobias Kötter, Thorsten Meinl, Peter Ohl, Kilian Thiel, and Bernd Wiswedel. KNIME — the Konstanz information miner. In *Data Analysis, Machine Learning and Applications*, Studies in Classification, Data Analysis, and Knowledge Organization, pages 319–326. Springer, 2009. doi: 10.1007/978-3-540-78246-9_38.

Yi-An Chen, Sethuraman Mani, Anastasia N. Nikolskaya, and Cathy H. Wu. Bioinformatics resource discovery and identifier mapping: Problems and approaches. *Briefings in Bioinformatics*, 6(3):239–250, 2005. doi: 10.1093/bib/6.3.239.

Edgar F. Codd. Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4):397–434, 1979. doi: 10.1145/320107.320109.

Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017. doi: 10.1038/nbt.3820.

Daniela Florescu, Alon Levy, Ioana Manolescu, and Dan Suciu. Query optimization in the presence of limited access patterns. In *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, pages 311–322, 1999. doi: 10.1145/304182.304210.

Hector Garcia-Molina, Yannis Papakonstantinou, Dallan Quass, Anand Rajaraman, Yehoshua Sagiv, Jeffrey Ullman, Vasilis Vassalos, and Jennifer Widom. The TSIMMIS approach to mediation: Data models and languages. *Journal of Intelligent Information Systems*, 8(2):117–132, 1997. doi: 10.1023/A:1008683107812.

Olaf Görlitz and Steffen Staab. SPLENDID: SPARQL endpoint federation exploiting VOID descriptions. In *Proceedings of the 2nd International Workshop on Consuming Linked Data (COLD 2011)*, volume 782 of *CEUR Workshop Proceedings*, 2011.

Laura M. Haas, Donald Kossmann, Edward L. Wimmers, and Jun Yang. Optimizing queries across diverse data sources. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB)*, pages 276–285, 1997.

Steve Harris and Andy Seaborne. SPARQL 1.1 query language. W3c recommendation, World Wide Web Consortium, March 2013. URL `https://www.w3.org/TR/sparql11-query/`. See in particular §4.2.2 (predicate–object and object lists) and the grammar rules for `ObjectList`, `Object`, and `GraphNode`.

Tomasz Imielinski and Witold Lipski. Incomplete information in relational databases. *Journal of the ACM*, 31(4):761–791, 1984. doi: 10.1145/1634.1886.

Jon Ison, Kristoffer Rapacki, Herve Menager, Matus Kalas, Emil Rydza, Piotr Chmura, Christian Anthon, Niall Beard, Karel Berka, Dan Bolser, et al. Tools and data services registry: A community effort to document bioinformatics resources. *Nucleic Acids Research*, 44(D1):D38–D47, 2016. doi: 10.1093/nar/gkv1116.

Nick Juty, Nicolas Le Novere, and Camille Laibe. Identifiers.org and MIRIAM Registry: Community resources to provide persistent identification. *Nucleic Acids Research*, 40(D1):D580–D586, 2012. doi: 10.1093/nar/gkr1097.

Minoru Kanehisa and Susumu Goto. KEGG: Kyoto encyclopedia of genes and genomes. *Nucleic Acids Research*, 28(1):27–30, 2000. doi: 10.1093/nar/28.1.27.

Minoru Kanehisa, Miho Furumichi, Yoko Sato, Masayuki Kawashima, and Mari Ishiguro-Watanabe. KEGG for taxonomy-based analysis of pathways and genomes. *Nucleic Acids Research*, 51(D1): D587–D592, 2023. doi: 10.1093/nar/gkac963.

Johannes Köster and Sven Rahmann. Snakemake — a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012. doi: 10.1093/bioinformatics/bts480.

Alon Y. Levy, Anand Rajaraman, and Joann J. Ordille. Querying heterogeneous information sources using source descriptions. In *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*, pages 251–262, 1996.

Leonid Libkin. SQL's three-valued logic and certain answers. *ACM Transactions on Database Systems*, 41(1):1:1–1:28, 2016. doi: 10.1145/2877206.

Yannis Papakonstantinou, Ashish Gupta, Hector Garcia-Molina, and Jeffrey D. Ullman. A query translation scheme for rapid implementation of wrappers. In *Deductive and Object-Oriented Databases (DOOD)*, volume 1013 of *Lecture Notes in Computer Science*, pages 161–186. Springer, 1995. doi: 10.1007/3-540-60608-4_39.

Eric Prud'hommeaux and Carlos Buil-Aranda. SPARQL 1.1 federated query. W3c recommendation, World Wide Web Consortium, March 2013. URL `https://www.w3.org/TR/sparql11-federated-query/`.

Nur Aini Rakhmawati, Jürgen Umbrich, Marcel Karnstedt, Ali Hasnain, and Michael Hausenblas. Querying over federated SPARQL endpoints: A state of the art survey. *CoRR*, abs/1306.1723, 2013.

RDFLib Team. RDFLib: A Python library for working with RDF. Software, 2026. URL `https://github.com/RDFLib/rdflib`. Version 7.6.0.

Muhammad Saleem, Yasar Khan, Ali Hasnain, Ivan Ermilov, and Axel-Cyrille Ngonga Ngomo. A fine-grained evaluation of SPARQL endpoint federation systems. *Semantic Web*, 7(5):493–518, 2016. doi: 10.3233/SW-150186.

Andreas Schwarte, Peter Haase, Katja Hose, Ralf Schenkel, and Michael Schmidt. FedX: Optimization techniques for federated query processing on linked data. In *The Semantic Web – ISWC 2011*, volume 7031 of *Lecture Notes in Computer Science*, pages 601–616. Springer, 2011. doi: 10.1007/978-3-642-25073-6_38.

Thomas Pellissier Tanon. Oxigraph: A graph database implementing the SPARQL standard. Software, 2026. URL `https://github.com/oxigraph/oxigraph`. Python bindings `pyoxigraph`, version 0.5.9.

Martijn P. van Iersel, Alexander R. Pico, Thomas Kelder, Jianjiong Gao, Isaac Ho, Kristina Hanspers, Bruce R. Conklin, and Chris T. Evelo. The BridgeDb framework: Standardized access to gene, protein and metabolite identifier mapping services. *BMC Bioinformatics*, 11:5, 2010. doi: 10.1186/1471-2105-11-5.

Gio Wiederhold. Mediators in the architecture of future information systems. *Computer*, 25(3): 38–49, 1992. doi: 10.1109/2.121508.

Sarala M. Wimalaratne, Nick Juty, John Kunze, Greg Janee, Julie A. McMurry, Niall Beard, Rafael Jimenez, Tim Clark, Sierra Moxon, Nicolas Le Novere, Henning Hermjakob, Christopher Mungall, and Michel Dumontier. Uniform resolution of compact identifiers for biomedical data. *Scientific Data*, 5:180029, 2018. doi: 10.1038/sdata.2018.29.

Katherine Wolstencroft, Robert Haines, Donal Fellows, Alan Williams, David Withers, Stuart Owen, Stian Soiland-Reyes, Ian Dunlop, Aleksandra Nenadic, Paul Fisher, Jiten Bhagat, Khalid Belhajjame, Finn Bacall, Alex Hardisty, Abraham Nieva de la Hidalga, Maria P. Balcazar Vargas, Shoaib Sufi, and Carole Goble. The Taverna workflow suite: Designing and executing workflows of web services on the desktop, web or in the cloud. *Nucleic Acids Research*, 41(W1):W557–W561, 2013. doi: 10.1093/nar/gkt328.

Ramana Yerneni, Chen Li, Hector Garcia-Molina, and Jeffrey Ullman. Computing capabilities of mediators. In *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, pages 443–454, 1999. doi: 10.1145/304182.304221.